

3.4 Guide d'installation et de déploiement local

Cette section détaille les procédures nécessaires pour configurer, exécuter et tester les environnements de développement de l'écosystème CYNA Defense (Backend, Frontend Web et Application Mobile).

3.4.1. Prérequis Systèmes

Avant toute manipulation, les outils suivants doivent être installés sur la machine de développement :

- **Node.js** (Version LTS recommandée)
- **npm** (Gestionnaire de paquets)
- **Git**
- Pour le mobile : L'application **Expo Go** sur smartphone (iOS/Android), ou un émulateur local (Android Studio / Xcode).

3.4.2. Installation de la Plateforme Web & API (Monorepo)

Ce dépôt centralise l'interface web (React/Vite) et la logique serveur (NestJS).

1. Clonage et installation des dépendances

Les dépendances doivent être installées de manière isolée pour chaque environnement.

```
git clone [URL_DE_REPO_GITHUB_WEB]
cd cyna-project

# Installation du Backend
cd cyna-backend
npm install

# Installation du Frontend
cd ../frontend-web
npm install
```

2. Configuration des Variables d'Environnement (.env)

Le frontend et le backend nécessitent leurs propres fichiers `.env`.

⚠ Règle de sécurité : Les clés secrètes (`SERVICE_KEY` , `STRIPE_SECRET` , mots de passe) ne doivent jamais être exposées dans le dossier frontend.

- **Backend (`cyna-backend/` .env)**

```
FRONTEND_URL=http://localhost:5173
SUPABASE_URL=https://vvqznavlkqjelpskjplh.supabase.co
SUPABASE_KEY=votre_cle_publique
SUPABASE_SERVICE_KEY=votre_cle_secrete_service_role
STRIPE_SECRET_KEY=rk_test_votre_cle_secrete
GROQ_API_KEY=votre_cle_api_groq
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=votre_email@gmail.com
SMTP_PASS=votre_mot_de_passe_app
```

- **Frontend (`frontend-web/` .env)** (Préfixe obligatoire : `VITE_`)

```
VITE_FRONTEND_URL=http://localhost:5173
VITE_SUPABASE_URL=https://vvqznavlkqjelpskjplh.supabase.co
VITE_SUPABASE_ANON_KEY=votre_cle_publique_anonyme
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_votre_cle_publique
```

3. Lancement des serveurs en local

L'écosystème nécessite le lancement des deux terminaux en parallèle :

- **Terminal 1 (Backend) :** `cd cyna-backend && npm run start:dev` (Écoute sur le port 3000)
- **Terminal 2 (Frontend) :** `cd frontend-web && npm run dev` (Écoute sur le port 5173)

3.4.3. Installation de l'Application Mobile (React Native / Expo)

L'application mobile dispose de son propre dépôt pour isoler son cycle de compilation.

1. Clonage et installation

Note : L'utilisation du flag `--legacy-peer-deps` est indispensable pour garantir la stabilité de l'environnement de test face aux exigences strictes de React 19.

```
git clone [URL_DE_REPO_GITHUB_MOBILE]
cd cyna-mobile-app
npm install --legacy-peer-deps
```

2. Variables d'environnement (`.env`) (Préfixe obligatoire : `EXPO_PUBLIC_`)

```
EXPO_PUBLIC_API_URL=https://votre-backend-render.com/api
EXPO_PUBLIC_SUPABASE_URL=https://vvqznavlkqjelpskjplh.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=votre_cle_anonyme_publique
EXPO_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_votre_cle_publique
```

3. Lancement en local

```
npx expo start
```

Action : Scanner le QR Code affiché dans le terminal avec l'application Expo Go, ou appuyer sur la touche `a` (Android) / `i` (iOS) pour utiliser un émulateur.

3.4.4. Procédures de Tests (QA) et Déploiement (CI/CD)

Chaque environnement possède son propre pipeline de validation et de déploiement automatisé :

1. Assurance Qualité (Tests Automatisés)

- **Backend (NestJS / Jest) :** `npm run test` et `npm run test:e2e`
- **Frontend Web (React / Vitest) :** `npx vitest run --environment jsdom`
- **Mobile (Jest & Mock Tests) :** `npm run test -- --verbose`

2. Déploiement Continu (CI/CD) & Cloud

- **Backend (Render) :** À chaque mise à jour sur la branche `main`, Render intercepte le code, compile le TypeScript (`npm run build`) et redémarre le serveur (`npm run start:prod`).

- **Frontend Web (Vercel)** : Vercel détecte les changements, exécute le build, et déploie le dossier `dist/` de manière distribuée.
- **Mobile (EAS Build)** : La compilation des exécutables est gérée dans le cloud par Expo Application Services.
 - *Commande de génération APK (Android)* :

```
eas build -p android --profile preview
```

3.4.5. Orchestration locale avec Docker (Alternative)

Pour garantir une stricte parité entre les environnements de développement et éviter les problèmes liés aux versions de Node.js, la plateforme (Backend + Frontend Web) peut être exécutée via des conteneurs Docker.

1. Prérequis supplémentaires

- **Docker Engine** et **Docker Compose** installés et en cours d'exécution sur votre machine.

2. Exécution via Docker Compose

Assurez-vous que vos fichiers `.env` sont bien configurés. Ensuite, depuis la racine du monorepo (où se trouve le fichier `docker-compose.yml`), lancez la commande suivante pour construire et démarrer les conteneurs en arrière-plan :

```
docker-compose up --build -d
```

3. Accès aux services et gestion

Une fois les conteneurs montés, les services sont accessibles sur leurs ports respectifs :

- **Backend API** : `http://localhost:3000`
- **Frontend Web** : `http://localhost:5173`

Pour stopper et détruire les conteneurs proprement une fois le développement terminé :

```
docker-compose down
```